

```

-- Exercício 1
CREATE OR REPLACE FUNCTION bloquear_exclusao()
RETURNS TRIGGER
AS $$
BEGIN
    IF OLD.quantidade > 0 THEN
        RAISE EXCEPTION 'Não é possível remover livros com exemplares
disponíveis no estoque.';
    END IF;

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_bloquear_exclusao
BEFORE DELETE ON livro
FOR EACH ROW
EXECUTE FUNCTION bloquear_exclusao();

-- Exercício 2
CREATE OR REPLACE FUNCTION log_exclusao_livro()
RETURNS TRIGGER
AS $$
BEGIN
    INSERT INTO log_livro (titulo, quantidade_antiga, quantidade_nova,
data_log)
    VALUES (
        OLD.titulo,
        OLD.quantidade,
        NULL,
        CURRENT_TIMESTAMP
    );

    RETURN OLD;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_log_exclusao
AFTER DELETE ON livro
FOR EACH ROW
EXECUTE FUNCTION log_exclusao_livro();

-- Exercício 3

```

```

CREATE OR REPLACE FUNCTION validar_limite_estoque()
RETURNS TRIGGER
AS $$
BEGIN
    IF NEW.quantidade > 100 THEN
        RAISE EXCEPTION 'Quantidade de estoque inválida. O limite máximo
permitido é de 100 exemplares.';
    END IF;

    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validar_limite
BEFORE UPDATE ON livro
FOR EACH ROW
EXECUTE FUNCTION validar_limite_estoque();

```

```

1  -- Exercício 1
2  CREATE OR REPLACE FUNCTION bloquear_exclusao()
3  RETURNS TRIGGER
4  AS $$
5  BEGIN
6      IF OLD.quantidade > 0 THEN
7          RAISE EXCEPTION 'Não é possível remover livros com exemplares disponíveis no estoque.';
8      END IF;
9
10     RETURN OLD;
11 END;
12 $$ LANGUAGE plpgsql;
13
14 CREATE TRIGGER trg_bloquear_exclusao
15 BEFORE DELETE ON livro
16 FOR EACH ROW
17 EXECUTE FUNCTION bloquear_exclusao();
18
19 -- Exercício 2
20 CREATE OR REPLACE FUNCTION log_exclusao_livro()
21 RETURNS TRIGGER
22 AS $$
23 BEGIN
24     INSERT INTO log_livro (titulo, quantidade_antiga, quantidade_nova, data_log)
25     VALUES (
26         OLD.titulo,
27         OLD.quantidade,
28         NULL,
29         CURRENT_TIMESTAMP
30     );
31     RETURN OLD;
32 END;
33 $$ LANGUAGE plpgsql;
34
35 CREATE TRIGGER trg_log_exclusao
36 AFTER DELETE ON livro
37 FOR EACH ROW
38 EXECUTE FUNCTION log_exclusao_livro();
39
40
41 -- Exercício 3
42 CREATE OR REPLACE FUNCTION validar_limite_estoque()
43 RETURNS TRIGGER
44 AS $$
45 BEGIN
46     IF NEW.quantidade > 100 THEN
47         RAISE EXCEPTION 'Quantidade de estoque inválida. O limite máximo permitido é de 100 exemplares.';
48     END IF;
49     RETURN NEW;
50 END;
51 $$ LANGUAGE plpgsql;
52
53 CREATE TRIGGER trg_validar_limite
54 BEFORE UPDATE ON livro
55 FOR EACH ROW
56 EXECUTE FUNCTION validar_limite_estoque();
57

```

```

12 $$ LANGUAGE plpgsql;
13
14 CREATE TRIGGER trg_bloquear_exclusao
15 BEFORE DELETE ON livro
16 FOR EACH ROW
17 EXECUTE FUNCTION bloquear_exclusao();
18
19 -- Exercício 2
20 CREATE OR REPLACE FUNCTION log_exclusao_livro()
21 RETURNS TRIGGER
22 AS $$
23 BEGIN
24     INSERT INTO log_livro (titulo, quantidade_antiga, quantidade_nova, data_log)
25     VALUES (
26         OLD.titulo,
27         OLD.quantidade,
28         NULL,
29         CURRENT_TIMESTAMP
30     );
31     RETURN OLD;
32 END;
33 $$ LANGUAGE plpgsql;
34
35 CREATE TRIGGER trg_log_exclusao
36 AFTER DELETE ON livro
37 FOR EACH ROW
38 EXECUTE FUNCTION log_exclusao_livro();
39
40 -- Exercício 3
41 CREATE OR REPLACE FUNCTION validar_limite_estoque()
42 RETURNS TRIGGER
43 AS $$
44 BEGIN
45     IF NEW.quantidade > 100 THEN
46         RAISE EXCEPTION 'Quantidade de estoque inválida. O limite máximo permitido é de 100 exemplares.';
47     END IF;
48     RETURN NEW;
49 END;
50 $$ LANGUAGE plpgsql;
51
52 CREATE TRIGGER trg_validar_limite
53 BEFORE UPDATE ON livro
54 FOR EACH ROW
55 EXECUTE FUNCTION validar_limite_estoque();
56
57
58

```